

Repaso para el examen de PDS

Patron anotaciones

```
1 package com.cafeteria.anotaciones;
2
3 import java.lang.annotation.ElementType;
4
5 /**
6  * Permite identificar clases que representan ingredientes.
7  */
8 @Retention(RetentionPolicy.RUNTIME)
9 @Target(ElementType.TYPE)
10 public @interface Ingrediente {
11     String name();
12     String tipo();
13 }
14
15 import com.cafeteria.anotaciones.Ingrediente;
16
17 @Ingrediente(name = "Café molido", tipo = "Ingrediente base")
18 public class CafeMolido
19     extends com.cafeteria.modelo.Ingrediente {
20
21     public CafeMolido() {
22         super(
23             "Café molido",
24             0.50,
25             20);
26     }
27 }
```

Patron Singleton

```
public class RegistroProcesador {
    private static RegistroProcesador instancia = null;
    private Map<String, Class> procesadores = new HashMap<>();

    private RegistroProcesador() {
    }

    public static RegistroProcesador getInstance() {
        if (instancia == null) {
            instancia = new RegistroProcesador();
        }
        return instancia;
    }

    public void registrarProcesador(String pkg) {
        try {
            ClassPath classPath = ClassPath.from(RegistroProcesador.class.getClassLoader());
            var clases = classPath.getTopLevelClassesRecursive(pkg);
            for (var it: clases) {
                var anot = it.load().getAnnotation(ProcesadorDocumento.class);
                if (anot != null) {
                    procesadores.put(anot.name(), it.load());
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```

    public Class<? extends Procesador> obtenerClase(String tipo) {
        return procesadores.get(tipo);
    }

    public void descubrirAnotadas() {
        procesadores.values().forEach(System.out::println);
    }
}

```

Patron factory

```

<> Java
public class ProductoFactory {

    public Producto crearProducto(String tipo) {

        if(tipo.equalsIgnoreCase("cafe"))
            return new Cafe();

        if(tipo.equalsIgnoreCase("capuchino"))
            return new Capuchino();

        if(tipo.equalsIgnoreCase("te"))
            return new Te();

        return null;
    }

}

```

Uso

```

<> Java
ProductoFactory factory = new ProductoFactory();

Producto producto = factory.crearProducto("cafe");

```

Patron Decorator("Se utilizó Decorator porque necesitaba añadir comportamientos al objeto de forma dinámica sin modificar la clase original ni crear múltiples combinaciones de clases. Cada decorador envuelve al objeto anterior y agrega una nueva funcionalidad.")

1 Componente (Interfaz)

Todos deben hacer lo mismo.

```
</> Java
public interface Procesador {

    void procesar();

}
```



2 Componente concreto

Hace el trabajo básico.

```
</> Java
public class ProcesadorDocumento
    implements Procesador{

    @Override
    public void procesar(){

        System.out.println("Procesando documento");

    }

}
```



3 Decorator (el que tienes)

Tu código es prácticamente el correcto.

```
</> Java
public abstract class ProcesadorDecorator
    implements Procesador{

    protected Procesador procesador;

    public ProcesadorDecorator(Procesador p){

        this.procesador = p;

    }

}
```



4 Decorador concreto

Por ejemplo Compresión.

```
</> Java
public class Compresion
    extends ProcesadorDecorador{

    public Compresion(Procesador p){

        super(p);

    }

    @Override
    public void procesar(){

        procesador.procesar();

        System.out.println("Aplicando compresión");

    }

}
```

Observa esta línea.

```
</> Java
procesador.procesar();
```

Patron Strategy("Porque el sistema necesita cambiar el comportamiento de forma dinámica. En lugar de usar muchos if o switch, cada algoritmo se implementa en una clase que sigue una misma interfaz. El contexto utiliza esa interfaz y puede cambiar de estrategia sin modificar su código.")

```
</> Java
public interface MetodoPago {

    void pagar(double monto);

}
```

2 Concrete Strategy

Cada estrategia implementa la interfaz.

Tarjeta

```
</> Java
public class PagoTarjeta implements MetodoPago{

    @Override
    public void pagar(double monto){

        System.out.println("Pagando con tarjeta $" + monto);

    }

}
```

3 Context

Es la clase que utiliza la estrategia.

```
</> Java
public class Pedido {

    private MetodoPago metodoPago;

    public Pedido(MetodoPago metodoPago){

        this.metodoPago = metodoPago;

    }

    public void procesarPago(double monto){

        metodoPago.pagar(monto);

    }

}
```

4 Main

Aquí eliges la estrategia.

```
</> Java
MetodoPago pago = new PagoTarjeta();

Pedido pedido = new Pedido(pago);

pedido.procesarPago(100);
```

Salida

Patron Builder ("Se utilizó Builder para construir objetos complejos paso a paso. Este patrón evita constructores con muchos parámetros, mejora la legibilidad del código y permite crear objetos con diferentes configuraciones de forma flexible.")

Java



```
public class CafeBuilder {

    private String tamaño;
    private boolean azucar;
    private boolean leche;

    public CafeBuilder tamaño(String tamaño){

        this.tamaño = tamaño;
        return this;

    }

    public CafeBuilder azucar(boolean azucar){

        this.azucar = azucar;
        return this;

    }

    public CafeBuilder leche(boolean leche){

        this.leche = leche;
        return this;

    }

}
```

```
    public Cafe build(){

        return new Cafe(
            tamaño,
            azucar,
            leche);

    }

}
```

3. Cliente (Main)

Java



```
public class Main {

    public static void main(String[] args) {

        Cafe cafe = new CafeBuilder()
            .tamaño("Grande")
            .leche(true)
            .azucar(true)
            .build();

        System.out.println(cafe);

    }

}
```

Patron Composite ("Se utilizó Composite porque era necesario representar una estructura jerárquica donde un objeto compuesto contiene otros objetos del mismo tipo. Tanto los elementos individuales (Leaf) como los grupos de elementos (Composite) implementan la misma interfaz, permitiendo que el cliente los trate de forma uniforme.")

1. Component (Interfaz)

Define las operaciones comunes.

```
</> Java
public interface Producto {
    void mostrar();
}
```

2. Leaf (Objeto individual)

Representa un elemento simple.

```
</> Java
public class Cafe implements Producto {

    @Override
    public void mostrar() {
        System.out.println("Café");
    }
}
```

```
</> Java
public class Combo implements Producto {

    private List<Producto> productos = new ArrayList<>();

    public void agregar(Producto producto) {
        productos.add(producto);
    }

    @Override
    public void mostrar() {
        for (Producto p : productos) {
            p.mostrar();
        }
    }
}
```

4. Cliente (Main)

```
</> Java
Combo combo = new Combo();

combo.agregar(new Cafe());
combo.agregar(new Pastel());

combo.mostrar();
```



Patron State ("El patrón State se utiliza cuando un objeto debe cambiar su comportamiento según su estado. En lugar de usar múltiples if o switch, cada estado se implementa en una clase distinta y el Contexto delega el comportamiento al estado actual.")

Java

```
public interface Estado {  
    void manejar();  
}
```

2. Estados concretos

Cada estado implementa la interfaz.

Java

```
public class EstadoPendiente implements Estado {  
  
    @Override  
    public void manejar() {  
        System.out.println("Pedido pendiente");  
    }  
}
```

Java

```
public class EstadoEnProceso implements Estado {  
  
    @Override  
    public void manejar() {  
        System.out.println("Pedido en proceso");  
    }  
}
```

3. Contexto

Es el objeto cuyo comportamiento cambia.

Java

```
public class Pedido {  
  
    private Estado estado;  
  
    public Pedido(Estado estado) {  
        this.estado = estado;  
    }  
  
    public void setEstado(Estado estado) {  
        this.estado = estado;  
    }  
  
    public void procesar() {  
        estado.manejar();  
    }  
}
```

4. Cliente

Java

```
Pedido pedido = new Pedido(new EstadoPendiente());  
  
pedido.procesar();  
  
pedido.setEstado(new EstadoEnProceso());  
  
pedido.procesar();  
  
pedido.setEstado(new EstadoEntregado());  
  
pedido.procesar();
```

Salida

```
Pedido pendiente  
Pedido en proceso  
Pedido entregado
```


Patrón Observer("El patrón Observer se utiliza cuando un objeto (Subject) necesita notificar automáticamente a varios objetos (Observers) sobre un cambio de estado. Los observadores se registran en el Subject y reciben la notificación sin que el Subject conozca su implementación concreta.")

Java

```
public interface Observer {  
    void actualizar();  
}
```

2. Observadores concretos

Cada uno hace algo diferente.

Java

```
public class CorreoObserver implements Observer {  
  
    @Override  
    public void actualizar() {  
        System.out.println("Enviando correo...");  
    }  
  
}
```

Java

```
import java.util.ArrayList;  
import java.util.List;  
  
public class Pedido {  
  
    private List<Observer> observers = new ArrayList<>();  
  
    public void agregarObserver(Observer o) {  
        observers.add(o);  
    }  
  
    public void eliminarObserver(Observer o) {  
        observers.remove(o);  
    }  
  
    public void notificar() {  
        for (Observer o : observers) {  
            o.actualizar();  
        }  
    }  
  
}
```

</> Java



```
public class Main {

    public static void main(String[] args) {

        Pedido pedido = new Pedido();

        pedido.agregarObserver(new CorreoObserver());
        pedido.agregarObserver(new SmsObserver());

        pedido.notificar();

    }

}
```

Salida

Enviando correo...
Enviando SMS...



Resumen de todos los patrones que has repasado



Patrón	Clases principales	Idea principal
Factory	Producto + Factory	Crear objetos
Builder	Producto + Builder (+ Director opcional)	Construir objetos paso a paso
Strategy	Interfaz + Estrategias + Contexto	Cambiar un comportamiento
Decorator	Componente + Decorador + Decoradores	Agregar funcionalidades
Composite	Component + Leaf + Composite	Tratar grupos y objetos individuales igual
State	State + Estados + Contexto	Cambiar comportamiento según el estado
Observer	Observer + Observadores + Subject + Cliente	Notificar automáticamente cambios

Patron Facade

"El patrón Facade proporciona una interfaz simple para acceder a un conjunto de clases o subsistemas complejos. El cliente interactúa únicamente con la fachada, que coordina las llamadas a los diferentes subsistemas."

</> Java



```
public class CalentarAgua {

    public void ejecutar() {
        System.out.println("Calentando agua");
    }

}
```

</> Java



```
public class MolerCafe {

    public void ejecutar() {
        System.out.println("Moliendo café");
    }

}
```

2 Facade

Aquí está el patrón.

```
Java
public class CafeteriaFacade {

    private CalentarAgua agua = new CalentarAgua();
    private MolerCafe molino = new MolerCafe();
    private PrepararCafe cafe = new PrepararCafe();

    public void prepararCafe() {

        agua.ejecutar();
        molino.ejecutar();
        cafe.ejecutar();

    }

}
```

3. Cliente ✓

```
Java
public class Main {

    public static void main(String[] args) {

        CafeteriaFacade fachada = new CafeteriaFacade();

        fachada.realizarPedido();

    }

}
```

Patron Adapter ("El patrón Adapter se utiliza para hacer compatibles dos interfaces que originalmente no lo son. El Adapter implementa la interfaz que espera el cliente (Target) y traduce las llamadas hacia la clase existente (Adaptee).")

```
Java
public interface Pago {

    void pagar();

}
```

2 Adaptee

Es la clase existente (la incompatible).

```
Java
public class PagoExterno {

    public void realizarPago() {
        System.out.println("Pago realizado");
    }

}
```

Observa que no tiene

3 Adapter (Aquí está el patrón)

```
</> Java
public class PagoAdapter implements Pago {

    private PagoExterno pagoExterno;

    public PagoAdapter(PagoExterno pagoExterno) {
        this.pagoExterno = pagoExterno;
    }

    @Override
    public void pagar() {
        pagoExterno.realizarPago();
    }

}
```

Aquí ocurre la adaptación.

4 Cliente

```
</> Java
public class Main {

    public static void main(String[] args) {

        Pago pago = new PagoAdapter(new PagoExterno());

        pago.pagar();

    }

}
```

Salida:

Patrón Command("El patrón Command encapsula una solicitud como un objeto. De esta forma, desacopla al objeto que solicita la acción (Invoker) del objeto que realmente la ejecuta (Receiver), permitiendo agregar funcionalidades como colas, historial o deshacer acciones.")

1 Command (Interfaz)

Define la acción.

```
</> Java
public interface Command {

    void ejecutar();

}
```

2 Receiver

Es quien sabe hacer el trabajo.

```
</> Java
public class Cafetera {

    public void prepararCafe() {
        System.out.println("Preparando café...");
    }

}
```

3 Concrete Command

Implementa la interfaz y usa al Receiver.

```
Java

public class PrepararCafeCommand implements Command {

    private Cafetera cafetera;

    public PrepararCafeCommand(Cafetera cafetera) {
        this.cafetera = cafetera;
    }

    @Override
    public void ejecutar() {
        cafetera.prepararCafe();
    }

}
```

4 Invoker

Es quien ejecuta el comando.

```
Java

public class Boton {

    private Command command;

    public Boton(Command command) {
        this.command = command;
    }

    public void presionar() {
        command.ejecutar();
    }

}
```

5 Cliente

```
Java

public class Main {

    public static void main(String[] args) {

        Cafetera cafetera = new Cafetera();

        Command comando =
            new PrepararCafeCommand(cafetera);

        Boton boton = new Boton(comando);

        boton.presionar();

    }

}
```

Salida

```
Preparando café...
```